



YOLOv10 Windows 설치 및 실행 가이드

Windows PC에서 YOLOv10을 이용한 객체 검출 시스템 구축

버전 1.0 | 2024년 업데이트



목차 (Table of Contents)

- [1. 개요](#)
- [2. 사전 준비](#)
- [3. Conda 환경 설정](#)
- [4. YOLOv10 모델 다운로드](#)
- [5. 소스코드 설명](#)
 - [5.1 단일 이미지 검출 \(101_predict_image.py\)](#)
 - [5.2 실시간 웹캠 검출 \(102_live_stream.py\)](#)
- [6. 실행 방법](#)
- [7. 문제 해결](#)
- [8. 추가 정보](#)



1. 개요

본 가이드는 Windows PC 환경에서 최신 객체 검출 모델인 **YOLOv10**을 설치하고 실행하는 방법을 단계별로 안내합니다. YOLOv10은 실시간 객체 검출에 최적화된 딥러닝 모델로, 높은 정확도와 빠른 처리 속도를 자랑합니다.

주요 기능

- 단일 이미지 객체 검출: JPG, PNG 등 정적 이미지에서 객체 인식
- 실시간 웹캠 검출: 웹캠을 통한 실시간 객체 추적
- 80개 클래스 지원: COCO 데이터셋 기반 다양한 객체 인식
- 고성능 처리: CPU/GPU 환경 모두 지원



참고사항

본 가이드는 Anaconda/Miniconda 환경을 기반으로 작성되었으며, Python 3.8 이상 버전이 필요합니다.



2. 사전 준비

1 시스템 요구사항

- 운영체제: Windows 10/11 (64비트)
- 메모리: 최소 8GB RAM (16GB 권장)
- 저장공간: 5GB 이상 여유 공간
- Python: 3.8 ~ 3.11 버전
- 웹캠: 실시간 검출을 위한 카메라 (선택사항)

2 필수 소프트웨어 설치

Anaconda 다운로드

<https://www.anaconda.com/download>

또는 Miniconda (경량 버전) 설치 가능

3 프로젝트 폴더 구조

다음과 같은 폴더 구조를 생성하세요:

```
yolov10_project/
├── 101_predict_image.py    # 단일 이미지 검출 스크립트
├── 102_live_stream.py      # 실시간 웹캠 검출 스크립트
├── weights/               # 모델 파일 저장 폴더
│   └── yolov10n.pt        # YOLOv10 Nano 모델
├── images/               # 테스트 이미지 폴더
│   └── class.jpg          # 테스트용 이미지 파일
└── yolov10_output_cv/     # 결과 이미지 저장 폴더 (자동 생성)
```

⚠️ 중요

폴더 구조를 정확히 맞춰야 스크립트가 정상 작동합니다. 특히 `weights`와 `images` 폴더는 필수입니다.

🔧 3. Conda 환경 설정

1 Conda 환경 생성 및 활성화

Anaconda Prompt 또는 Command Prompt를 관리자 권한으로 실행한 후 다음 명령어를 순서대로 입력하세요:

```
# 새로운 Python 3.10 환경 생성
conda create -n yolov10_env python=3.10 -y

# 환경 활성화
conda activate yolov10_env
```

2 PyTorch 설치

🖥️ CPU vs GPU

GPU가 없거나 CUDA 설정이 복잡한 경우 CPU 버전을 사용하세요. 성능은 다소 떨어지지만 정상 작동합니다.

CPU 버전 (권장 - 초보자용)

```
conda install pytorch torchvision torchaudio cpuonly -c pytorch
```

GPU 버전 (NVIDIA GPU + CUDA 11.8)

```
conda install pytorch torchvision torchaudio pytorch-cuda=11.8 -c pytorch -c nvidia
```

3 필수 라이브러리 설치

```
# YOLOv10 및 관련 라이브러리 설치
pip install ultralytics

# OpenCV 및 기타 라이브러리 설치
pip install opencv-python numpy matplotlib

# 설치 확인
python -c "import cv2, torch, ultralytics; print('✅ 모든 라이브러리 설치 완료')"
```

🔍 설치 확인

마지막 명령어에서 오류가 발생하지 않으면 모든 라이브러리가 정상 설치된 것입니다.

📦 4. YOLOv10 모델 다운로드

1 weights 폴더 생성

```
# 프로젝트 루트 디렉토리에서 실행
mkdir weights
```

2 모델 파일 다운로드

🔍 YOLOv10n.pt 다운로드

<https://github.com/THU-MIG/yolov10/releases/download/v1.1/yolov10n.pt>

파일 크기: 약 5.8MB | 'n'은 Nano 버전 (가장 가벼운 모델)

다운로드한 `yolov10n.pt` 파일을 `weights` 폴더 안에 저장하세요.

- **yolov10n.pt**: 가장 빠름, 정확도 보통 (권장)
- **yolov10s.pt**: 균형잡힌 성능
- **yolov10m.pt**: 높은 정확도, 속도 보통
- **yolov10l.pt**: 매우 높은 정확도, 속도 느림

5. 소스코드 설명

5.1 단일 이미지 검출 (101_predict_image.py)

이 스크립트는 정적 이미지 파일에서 객체를 검출하고 결과를 시각화합니다.

```
from ultralytics import YOLO
import cv2
import os

# --- 설정 (Configuration) ---
# 1. 모델 로드 (가중치 파일 경로를 수정)
# 'yolov10n.pt' 파일이 현재 경로의 './weights/' 폴더에 있다고 가정합니다.
model_name = './weights/yolov10n.pt'
try:
    model = YOLO(model_name)
    print(f"모델 로드 완료: {model_name}")
except FileNotFoundError:
    print(f"❌ 오류: 모델 파일 '{model_name}'을 찾을 수 없습니다. 경로를 확인하거나 수동으로 다운로드하세요.")
    exit()

# 사용할 이미지 파일 경로
image_path = './images/class.jpg'
if not os.path.exists(image_path):
    print(f"❌ 오류: 이미지 파일 '{image_path}'을 찾을 수 없습니다. 경로를 확인하세요.")
    exit()

# --- 2. 이미지 불러오기 및 추론 실행 ---
print(f"이미지 경로: {image_path}")
img = cv2.imread(image_path)
if img is None:
    print(f"❌ 오류: OpenCV로 이미지를 로드할 수 없습니다.")
    exit()

print("추론 시작...")
# predict 메소드를 실행하되, save=True는 더 이상 사용하지 않습니다.
results = model.predict(
    source=img, # 이미지를 직접 전달
    conf=0.25,
    verbose=False # 출력 간소화
)
print("추론 완료.")

# --- 3. 결과 시각화 (OpenCV 사용) ---
if results:
    result = results[0] # 첫 번째 이미지 결과
    num_detections = len(result.boxes)
    print(f"\n✅ 검출된 총 객체 수: {num_detections}")

    # 검출된 결과에서 바운딩 박스, 클래스 ID, 신뢰도 추출
    boxes = result.boxes.xyxy.cpu().numpy() # 바운딩 박스 좌표 (x1, y1, x2, y2)
    classes = result.boxes.cls.cpu().numpy().astype(int) # 클래스 ID
    confidences = result.boxes.conf.cpu().numpy() # 신뢰도

    # 시각화 설정
    thickness = 2
    font = cv2.FONT_HERSHEY_SIMPLEX
    font_scale = 0.7

    # 원본 이미지에 바운딩 박스 및 레이블 그리기
    for box, cls_id, conf in zip(boxes, classes, confidences):
        x1, y1, x2, y2 = map(int, box)
        class_name = model.names[cls_id]

        # 색상 설정 (클래스 ID 기반으로 고유한 색상 생성)
        color = (255, 0, 0) # tuple(int(c) for c in model.model.colors[cls_id])

        # 바운딩 박스 그리기
        cv2.rectangle(img, (x1, y1), (x2, y2), color, thickness)
```

```

        # 레이블 텍스트 생성: "클래스 이름 (신뢰도%)"
        label = f"{class_name} ({conf:.2f})"

        # 텍스트 크기 계산
        (w, h), _ = cv2.getTextSize(label, font, font_scale, thickness)

        # 텍스트 배경을 위한 사각형 그리기
        # 텍스트를 바운딩 박스 위에 표시
        cv2.rectangle(img, (x1, y1 - h - 5), (x1 + w, y1), color, -1)

        # 텍스트 그리기
        cv2.putText(img, label, (x1, y1 - 5), font, font_scale, (255, 255, 255), thickness, cv2.LINE_AA)

        print(f"   클래스: {class_name}, 신뢰도: {conf:.2f}, 좌표: ({x1}, {y1})~({x2}, {y2})")

# 4. 결과 이미지 화면에 표시
cv2.imshow('YOLOv10 Custom Detection', img)
cv2.waitKey(0)
cv2.destroyAllWindows()

# (선택 사항) 결과를 파일로 저장
output_dir = 'yolov10_output_cv'
os.makedirs(output_dir, exist_ok=True)
output_path = os.path.join(output_dir, os.path.basename(image_path).replace('.', '_cv_output.'))
cv2.imwrite(output_path, img)
print(f"\n결과 이미지는 다음 경로에 저장되었습니다: \n   -> {output_path}")

else:
    print("❌ 검출된 객체가 없습니다.")

```

🔍 코드 주요 구성 요소

- **모델 로드:** YOLO 클래스로 사전 훈련된 모델 불러오기
- **이미지 처리:** OpenCV로 이미지 파일 읽기
- **객체 검출:** model.predict()로 추론 실행
- **결과 시각화:** 바운딩 박스와 레이블 그리기
- **결과 저장:** 검출 결과가 포함된 이미지 파일 저장

5.2 실시간 웹캠 검출 (102_live_stream.py)

이 스크립트는 웹캠을 통해 실시간으로 객체를 검출합니다.

```

from ultralytics import YOLO
import cv2
import os

# --- 1. 설정 (Configuration) ---
# 'yolov10n.pt' 파일이 현재 경로의 './weights/' 폴더에 있다고 가정합니다.
model_name = './weights/yolov10n.pt'
try:
    # 💡 모델 로드
    model = YOLO(model_name)
    print(f"✅ 모델 로드 완료: {model_name}")
except FileNotFoundError:
    print(f"❌ 오류: 모델 파일 '{model_name}'을 찾을 수 없습니다. 경로를 확인하거나 수동으로 다운로드하세요.")
    exit()

# 💡 클래스 ID에 따른 고유한 색상을 생성하는 함수 정의
def get_color_by_class(cls_id):
    """클래스 ID에 따라 고유한 BGR 색상 튜플을 반환합니다."""
    # [B, G, R] 순서로 20개의 색상을 미리 정의합니다.
    colors = [
        (255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0), (0, 255, 255), (255, 0, 255),
        (128, 0, 0), (0, 128, 0), (0, 0, 128), (128, 128, 0), (0, 128, 128), (128, 0, 128),
        (255, 128, 0), (255, 0, 128), (0, 255, 128), (128, 255, 0), (128, 0, 255), (0, 128, 255),
        (255, 255, 128), (128, 255, 255)
    ]
    return colors[cls_id % len(colors)]

# --- 2. 웹캠 캡처 객체 초기화 ---
# 0번은 보통 기본 웹캠을 의미합니다. 여러 대의 카메라가 연결된 경우 1, 2 등으로 변경하세요.
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("❌ 오류: 카메라를 열 수 없습니다. 카메라가 연결되어 있는지 또는 인덱스(0)가 올바른지 확인하세요.")
    exit()

# --- 3. 실시간 프레임 처리 루프 ---
print("🚀 웹캠 실시간 객체 검출을 시작합니다. 'q' 키를 누르면 종료됩니다.")
#

```

```

while cap.isOpened():
    # 프레임 읽기
    success, frame = cap.read()

    if not success:
        print("프레임을 받을 수 없습니다 (스트림 끝?). 종료합니다.")
        break

    # 추론 실행 (프레임마다 실행)
    results = model.predict(
        source=frame, # 현재 프레임을 직접 전달
        conf=0.25,
        verbose=False,
        stream=True # 실시간 스트림 처리를 최적화
    )

    # 결과 시각화
    for result in results: # stream=True인 경우, results는 제너레이터입니다.

        # 검출된 결과에서 바운딩 박스, 클래스 ID, 신뢰도 추출
        boxes = result.bboxes.xyxy.cpu().numpy() # 바운딩 박스 좌표
        classes = result.bboxes.cls.cpu().numpy().astype(int) # 클래스 ID
        confidences = result.bboxes.conf.cpu().numpy() # 신뢰도

        # 시각화 설정
        thickness = 2
        font = cv2.FONT_HERSHEY_SIMPLEX
        font_scale = 0.7

        # 현재 프레임에 바운딩 박스 및 레이블 그리기
        for box, cls_id, conf in zip(boxes, classes, confidences):
            x1, y1, x2, y2 = map(int, box)
            class_name = model.names[cls_id]

            # 색상 설정 (클래스 ID 기반으로 고유한 색상 생성)
            color = get_color_by_class(cls_id)

            # 바운딩 박스 그리기
            cv2.rectangle(frame, (x1, y1), (x2, y2), color, thickness)

            # 레이블 텍스트 생성: "클래스 이름 (신뢰도%)"
            label = f"{class_name} ({conf:.2f})"

            # 텍스트 크기 계산
            (w, h), _ = cv2.getTextSize(label, font, font_scale, thickness)

            # 텍스트 배경을 위한 사각형 그리기
            cv2.rectangle(frame, (x1, y1 - h - 5), (x1 + w, y1), color, -1)

            # 텍스트 그리기
            cv2.putText(frame, label, (x1, y1 - 5), font, font_scale, (255, 255, 255), thickness, cv2.LINE_AA)

            # 실시간 출력에서는 print를 최소화합니다.
            # print(f" - 클래스: {class_name}, 신뢰도: {conf:.2f}")

        # 화면에 프레임 표시
        cv2.imshow('YOLOv10 Live Detection', frame)

        # 'q' 키를 누르면 루프 종료
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

# --- 4. 종료 및 자원 해제 ---
cap.release()
cv2.destroyAllWindows()
print("\n🎉 실시간 검출을 종료합니다.")

```

👤 실시간 처리 주요 특징

- **웹캠 초기화:** cv2.VideoCapture(0)로 기본 카메라 접근
- **무한 루프:** while문으로 연속적인 프레임 처리
- **스트림 최적화:** stream=True 옵션으로 성능 향상
- **색상 구분:** 클래스별 고유 색상으로 시각적 구분
- **실시간 종료:** 'q' 키로 언제든지 종료 가능



6. 실행 방법

1 테스트 이미지 준비

images 폴더에 class.jpg 파일을 저장하세요. 없는 경우 인터넷에서 다양한 객체가 포함된 이미지를 다운로드하여 사용할 수 있습니다.



테스트 이미지 추천

사람, 자동차, 동물 등이 포함된 일상적인 사진이 검출 테스트에 적합합니다.

2 단일 이미지 검출 실행

```
# Conda 환경 활성화
conda activate yolov10_env
```

```
# 프로젝트 폴더로 이동
cd path/to/yolov10_project
```

```
# 스크립트 실행
python 101_predict_image.py
```



실행 결과

- 콘솔에 검출된 객체 정보 출력
- OpenCV 창에 바운딩 박스가 그려진 이미지 표시
- 아무 키나 누르면 창 닫기
- yolov10_output_cv 폴더에 결과 이미지 자동 저장

3 실시간 웹캠 검출 실행

```
# 웹캠 검출 스크립트 실행
python 102_live_stream.py
```



실행 결과

- 웹캠 화면에 실시간 객체 검출 결과 표시
- 검출된 객체에 색상별 바운딩 박스 표시
- 클래스명과 신뢰도 점수 실시간 표시
- 'q' 키를 눌러 종료



주의사항

실시간 검출 시 CPU 사용률이 높아질 수 있습니다. 성능이 부족한 경우 신뢰도 임계값(conf)을 높이거나 해상도를 낮춰보세요.



7. 문제 해결



자주 발생하는 오류



FileNotFoundError: yolov10n.pt 파일을 찾을 수 없습니다

- weights 폴더가 존재하는지 확인
- 모델 파일이 정확한 위치에 있는지 확인
- 파일명이 정확한지 확인 (대소문자 구분)



카메라를 열 수 없습니다

- 웹캠이 다른 프로그램에서 사용 중인지 확인
- 카메라 인덱스를 1 또는 2로 변경해보기
- 카메라 드라이버 업데이트
- Windows 개인정보 설정에서 카메라 액세스 허용



ModuleNotFoundError: No module named 'ultralytics'

- Conda 환경이 활성화되었는지 확인
- pip install ultralytics 재실행
- Python 환경 변수 확인

⚡ 성능 최적화 팁

🚀 성능 향상 방법

- 신뢰도 임계값 조정: `conf=0.5`로 높여서 불필요한 검출 줄이기
- 이미지 크기 조정: 큰 이미지는 리사이즈하여 처리
- GPU 사용: CUDA 지원 GPU가 있다면 GPU 버전 설치
- 배치 처리: 여러 이미지를 한 번에 처리

🔍 디버깅 체크리스트

환경 확인

```
conda list | findstr torch
conda list | findstr ultralytics
conda list | findstr opencv
```

Python 경로 확인

```
python -c "import sys; print(sys.executable)"
```

라이브러리 импорт 테스트

```
python -c "import torch, cv2, ultralytics; print('모든 라이브러리 정상')"
```

📚 8. 추가 정보

🎯 YOLOv10에서 검출 가능한 객체 클래스

COCO 데이터셋 기반 80개 클래스를 지원합니다:

주요 카테고리

- 사람 및 동물: person, cat, dog, horse, sheep, cow, bird 등
- 교통수단: car, motorcycle, airplane, bus, train, truck, boat 등
- 일상용품: bottle, cup, fork, knife, spoon, bowl, laptop, mouse 등
- 스포츠: frisbee, skis, snowboard, sports ball, baseball bat 등
- 가구: chair, couch, bed, dining table, toilet, tv, refrigerator 등

⚙️ 고급 설정 옵션

`model.predict()` 주요 매개변수

- **conf**: 신뢰도 임계값 (0.0~1.0, 기본값 0.25)
- **iou**: NMS IoU 임계값 (기본값 0.45)
- **imgsz**: 입력 이미지 크기 (기본값 640)
- **device**: 실행 장치 ('cpu' 또는 'cuda')
- **max_det**: 최대 검출 개수 (기본값 300)

🔗 유용한 링크

- [YOLOv10 공식 GitHub](#)
- [Ultralytics 공식 문서](#)
- [PyTorch 설치 가이드](#)
- [OpenCV 공식 사이트](#)

💡 다음 단계 YOLOv10을 성공적으로 설치하셨다면, 다음과 같은 고급 기능도 시도해보세요:

- 커스텀 데이터셋으로 모델 훈련
- 비디오 파일 객체 검출
- 웹 애플리케이션과 통합
- 실시간 객체 추적 (tracking)

YOLOv10 Windows 설치 및 실행 가이드

본 문서는 교육 목적으로 제작되었으며, 최신 정보는 공식 문서를 참조하시기 바랍니다.